

**MODUL PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBYEK**

**MODUL 3
Encapsulation, Access Modifier, Getter–Setter, dan Validasi Data**



Disusun oleh:

Nirwana Haidar Hari, S.Pd., M.Kom.

**POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
PROGRAM STUDI DI LUAR KAMPUS UTAMA SUMENEP
PENS PSDKU SUMENEP**

2026

MODUL PRAKTIKUM P3

Encapsulation, Access Modifier, Getter–Setter, dan Validasi Data

1. Tujuan Praktikum

Setelah menyelesaikan praktikum, mahasiswa mampu:

1. Menjelaskan tujuan **encapsulation** dan **information hiding**.
2. Menggunakan private, public, dan access modifier secara tepat.
3. Membatasi akses langsung terhadap attribute.
4. Membuat getter dan setter.
5. Menambahkan validasi data melalui method.
6. Melakukan refactoring class dari hasil P2.
7. Melaksanakan *Agile Sprint* sederhana pada pengembangan proyek.

2. Review P2

Pada P2 mahasiswa telah membuat:

- class,
- object,
- attribute,
- method,
- constructor.

Contoh:

```
class Mahasiswa {  
  
    String nrp;  
    String nama;  
    double ipk;  
}
```

Permasalahan:

```
Mahasiswa mhs = new Mahasiswa();  
  
mhs.ipk = -10;
```

Nilai tersebut tetap dapat dimasukkan karena attribute dapat diakses secara langsung.

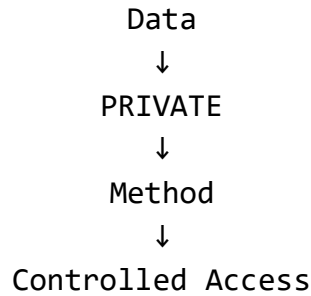
Pada P3 struktur tersebut akan diperbaiki menggunakan **encapsulation**.

3. Konsep Singkat

A. Encapsulation

Encapsulation merupakan mekanisme membungkus data dan perilaku dalam sebuah class serta membatasi akses langsung terhadap data.

Prinsip sederhana:



Contoh:

```
class Mahasiswa {  
  
    private String nrp;  
    private String nama;  
    private double ipk;  
  
}
```

Attribute tidak dapat lagi diubah secara langsung dari luar class.

B. Access Modifier

Access modifier menentukan tingkat akses terhadap attribute dan method.

Modifier	Akses
private	Hanya dari class yang sama
default	Dalam package yang sama
protected	Package dan subclass
public	Dapat diakses dari mana saja

Pada P3 fokus utama:

```
private  
public
```

C. Getter

Getter digunakan untuk mengambil nilai attribute.

```
public String getNama() {  
    return nama;  
}
```

Pemanggilan:

```
System.out.println(  
    mahasiswa.getNama()  
);
```

D. Setter

Setter digunakan untuk mengubah nilai attribute secara terkontrol.

```
public void setName(String nama) {
    this.nama = nama;
}
```

E. Validasi Data

Setter dapat digunakan untuk melakukan validasi.

Contoh:

```
public void setIpk(double ipk) {

    if (ipk >= 0 && ipk <= 4) {
        this.ipk = ipk;
    } else {
        System.out.println(
            "IPK tidak valid"
        );
    }
}
```

Dengan mekanisme tersebut:

```
mahasiswa.setIpk(-10);
```

tidak langsung mengubah data object.

4. Refactoring Class P2

Contoh sebelum P3:

```
class Buku {

    String kode;
    String judul;
    int tahunTerbit;
}
```

Refactoring:

```
class Buku {

    private String kode;
    private String judul;
    private int tahunTerbit;

    public Buku(
        String kode,
        String judul,
        int tahunTerbit
    ) {

        this.kode = kode;
```

```

        this.judul = judul;

        setTahunTerbit(
            tahunTerbit
        );
    }

    public String getKode() {
        return kode;
    }

    public String getJudul() {
        return judul;
    }

    public void setJudul(
        String judul
    ) {

        if (
            judul != null &&
            !judul.isEmpty()
        ) {
            this.judul = judul;
        }
    }

    public int getTahunTerbit() {
        return tahunTerbit;
    }

    public void setTahunTerbit(
        int tahunTerbit
    ) {

        if (tahunTerbit > 0) {
            this.tahunTerbit =
                tahunTerbit;
        }
    }
}

```

5. Agile Sprint – P3

Sprint Goal

Memperbaiki struktur class proyek dengan menerapkan encapsulation sehingga data object hanya dapat diakses dan diubah melalui mekanisme yang terkontrol.

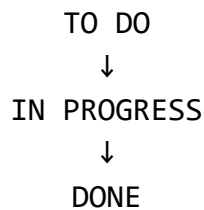
Sprint Backlog

Gunakan class hasil P2.

Contoh:

ID	Sprint Backlog	Status
SB-01	Mengubah attribute menjadi private	To Do
SB-02	Membuat getter	To Do
SB-03	Membuat setter	To Do
SB-04	Menambahkan validasi data	To Do
SB-05	Memperbaiki constructor	To Do
SB-06	Melakukan pengujian object	To Do

Status:



6. Tugas Praktikum

Tugas P3 – Refactoring Project dengan Encapsulation

Gunakan project dan source code P2.

Bagian A – Audit Class P2

Pilih minimal 3 class.

Buat tabel:

Class	Attribute	Kondisi P2	Perbaikan P3
Buku	kode	default	private
Buku	judul	default	private + setter
Buku	tahun	default	private + validation

Identifikasi attribute yang:

- hanya perlu dibaca,
- dapat diubah,
- membutuhkan validasi.

7. Bagian B – Encapsulation

Semua attribute utama harus diubah menjadi:

private

Contoh:

```
private String kode;  
private String nama;  
private double harga;  
Dilarang menggunakan:  
object.harga = 10000;  
dari luar class.
```

8. Bagian C – Getter dan Setter

Buat getter untuk attribute yang perlu dibaca.

Contoh:

```
public double getHarga() {  
    return harga;  
}
```

Buat setter hanya jika data memang boleh diubah.

Contoh:

```
public void setHarga(  
    double harga  
) {  
  
    if (harga > 0) {  
        this.harga = harga;  
    }  
}
```

Tidak semua attribute harus memiliki setter.

Contoh:

```
private String kode;
```

Jika kode tidak boleh berubah setelah object dibuat, cukup:

```
public String getCode() {  
    return kode;  
}
```

9. Bagian D – Validasi

Minimal terdapat **3 aturan validasi** pada project.

Contoh:

Harga

```
if (harga > 0)
```

Tahun

```
if (  
    tahun >= 2000 &&  
    tahun <= 2100
```

```

    )
    Nama
    if (
        nama != null &&
        !nama.isBlank()
    )

```

10. Bagian E – Constructor

Perbaiki constructor agar menggunakan aturan validasi.

Contoh:

```

    public Produk(
        String kode,
        String nama,
        double harga
    ) {

        this.kode = kode;
        setNama(nama);
        setHarga(harga);
    }

```

Dengan demikian constructor tidak mengabaikan aturan validasi.

11. Bagian F – Pengujian

Buat minimal dua jenis pengujian.

Test Valid

```

    Produk produk =
        new Produk(
            "P001",
            "Laptop",
            7000000
        );

```

Expected:

Object berhasil dibuat.

Test Invalid

```

    produk.setHarga(-5000);

```

Expected:

Harga tidak valid.

Data object tidak boleh berubah menjadi nilai yang tidak valid.

12. Luaran Tugas

Kumpulkan:

```

P3_NRP_Nama/
|
├── README.pdf
|
└── src/
    ├── Main.java
    ├── Class1.java
    ├── Class2.java
    └── Class3.java

```

README.pdf maksimal **3 halaman**.

Halaman 1

- Nama project
- Sprint Goal
- Sprint Backlog
- Tabel audit class P2

Halaman 2

Tampilkan tabel:

Class	Private Attribute	Getter	Setter	Validation
Class 1	...	...	...	...
Class 2	...	...	...	...
Class 3	...	...	...	...

Halaman 3

- screenshot program,
- test valid,
- test invalid,
- Sprint Review,
- Sprint Retrospective.

13. Sprint Review

Isi setelah program selesai.

Item	Hasil
Attribute berhasil di-private	...
Getter berjalan	...
Setter berjalan	...
Validasi berhasil	...
Constructor diperbaiki	...

Test invalid berhasil	...
Program dapat dijalankan	...

14. Sprint Retrospective

Jawab singkat.

What Went Well?

Apa yang berjalan baik?

What Went Wrong?

Apa kendala dalam melakukan refactoring?

Improvement

Apa yang akan diperbaiki pada sprint berikutnya?

Maksimal 3–5 kalimat.

15. Definition of Done

Tugas P3 dinyatakan selesai jika:

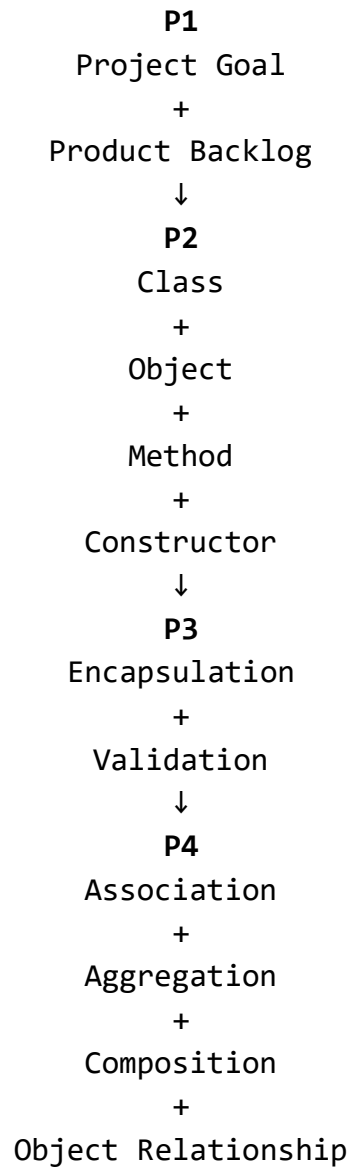
- Menggunakan project P2.
- Minimal 3 class direfactor.
- Attribute utama menggunakan private.
- Getter dibuat sesuai kebutuhan.
- Setter hanya dibuat bila diperlukan.
- Minimal 3 validasi data diterapkan.
- Constructor mengikuti aturan validasi.
- Tidak ada perubahan attribute secara langsung dari Main.
- Terdapat pengujian data valid.
- Terdapat pengujian data tidak valid.
- Program berhasil dikompilasi.
- Program berhasil dijalankan.
- Sprint Backlog diperbarui.
- Sprint Review tersedia.
- Sprint Retrospective tersedia.

16. Penilaian

Komponen	Bobot
Audit dan refactoring class P2	15%
Penerapan encapsulation	25%
Getter dan setter	15%
Validasi data	20%
Pengujian program	15%
Agile Sprint dan dokumentasi	10%
Total	100%

17. Target Sprint Berikutnya

Hasil P3 menjadi dasar untuk **P4 – Relasi Antarobject.**



Pada P4, class yang selama ini masih berdiri sendiri mulai **dihubungkan menjadi sebuah sistem** melalui relasi antarobject.